Taller 5

Shary Alejandra Velasquez – Luisa Fernanda Parra

Actividad 1 – Diagnóstico de relaciones

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	Un estudiante solicita préstamos; cada préstamo pertenece a un único estudiante. El préstamo necesita saber quién tiene el ejemplar.	Sí. El estudiante existe antes, durante y después de sus préstamos. Eliminar un préstamo no elimina al estudiante.	Asociación Estudiante 1 — 0..* Prestamo
Libro – Ejemplar	Un libro tiene una o varias copias físicas. Cada ejemplar es una copia de un libro concreto y nunca cambia de libro.	El libro puede catalogarse sin copias; el ejemplar no tiene significado sin la obra que representa.	Composición Libro 1 ◆— 1..* Ejemplar
Préstamo – Renovación	Cada renovación extiende el plazo de un préstamo y queda registrada en su historial.	No. Una renovación sin préstamo no significa nada.	Composición Prestamo 1 ◆— 0..* Renovacion
Usuario – Estudiante	Un estudiante es un tipo de usuario de SmartLibrary: se identifica, tiene nombre y correo, y opera en el sistema.	es un mismo objeto visto con dos niveles de abstracción.	herencia
Usuario – Bibliotecario	Un bibliotecario también es un usuario del sistema: se identifica y registra o gestiona préstamos.	No aplica, por la misma razón.	herencia

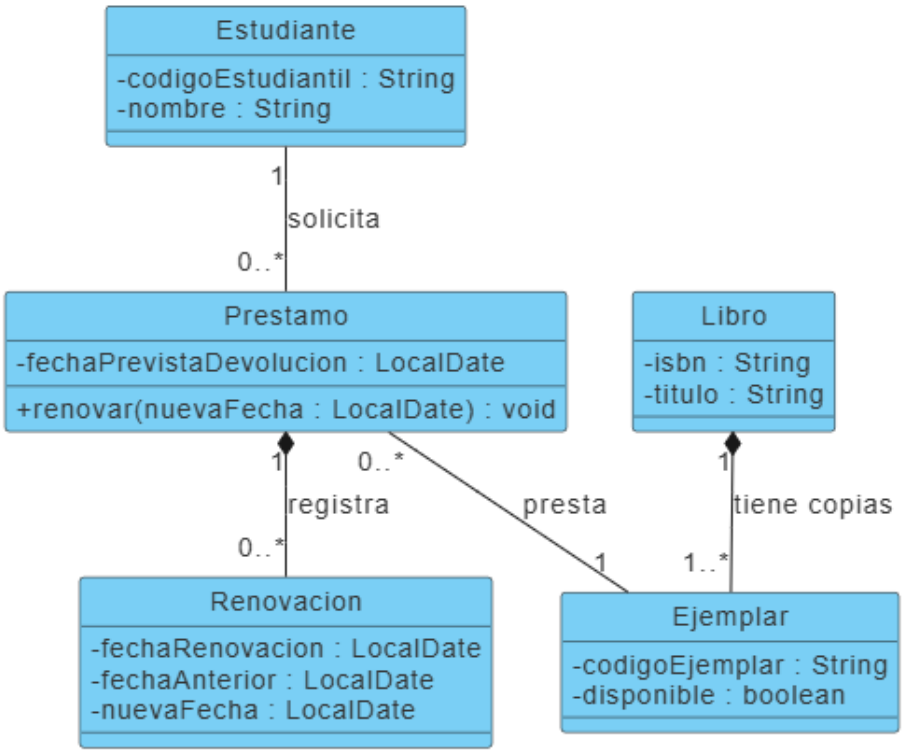
¿Que dos clases se relacionen significa necesariamente que una sea parte de la otra?
No. La mayoría de las relaciones son simplemente de asociación: el préstamo conoce al estudiante, pero el estudiante no es una parte del préstamo. Solo hay todo–parte cuando el dominio lo afirma, y aun entonces hay que distinguir si la parte puede vivir sin el todo (agregación) o no (composición).

¿Qué evidencia del requisito utilizaron para decidir?
el estudiante se matricula en la biblioteca antes de pedir libros y sigue existiendo al devolverlos; el ejemplar vuelve al estante después de cada préstamo; la renovación se registra sobre un préstamo concreto y no tiene sentido fuera de él y tanto estudiantes como bibliotecarios se identifican en el sistema con identificación, nombre y correo.

¿Cuál de las relaciones anteriores sería incorrecto decidir únicamente mirando los nombres de las clases?
Libro – Ejemplar. Por el nombre parece obvio un «tiene», pero la elección entre asociación, agregación y composición depende de una regla que los requisitos no aclaran, qué pasa con las copias cuando el título se retira del catálogo.

Actividad 2 – Asociación, agregación y composición

Relación	Tipo seleccionado	Justificación basada en el dominio
Estudiante – Prestamo	Asociación 1 — 0..*	el préstamo necesita saber quién tiene el ejemplar y ninguno controla la existencia del otro. Un estudiante puede no tener préstamos (0..*) y cada préstamo tiene exactamente un estudiante (1).
Prestamo – Ejemplar	Asociación 0..* — 1	El préstamo usa temporalmente un ejemplar que existe antes y después de él y que vuelve al estante. Un préstamo presta exactamente un ejemplar; un ejemplar acumula muchos préstamos en su historial (0..*), aunque solo uno activo a la vez
Libro – Ejemplar	Composición 1 ◆— 1..*	Sí hay todo–parte: el ejemplar es una copia física de una obra y no tiene significado sin ella; nace asociado a ese libro y nunca cambia de libro.
Prestamo – Renovacion	Composición 1 ◆— 0..*	La renovación pertenece a un préstamo concreto, nace dentro de él (en renovar) y si el préstamo desaparece su historial de renovaciones deja de tener significado. Un préstamo puede no haberse renovado nunca (0..*).



Actividad 3 – Herencia: ¿es-un o solo se parecen?

¿Estudiante es realmente un tipo de Usuario dentro de SmartLibrary?

Sí. En SmartLibrary el estudiante se identifica, tiene un correo institucional y opera en el sistema (consulta, pide préstamos, reserva). Cualquier operación pensada para un Usuario como: buscarlo por identificación, mostrar su nombre, enviarle un correo, funciona igual con un Estudiante. Además de los datos, comparte el significado: es alguien que usa la biblioteca a través del sistema.

¿Bibliotecario es realmente un tipo de Usuario?
Sí, dentro de SmartLibrary. El bibliotecario también se identifica y opera en el sistema, aunque con otras responsabilidades: registra y gestiona préstamos. La generalización es válida porque ambos son usuarios del sistema, no porque tengan atributos en común.

¿La existencia de identificacion, nombre y correo repetidos sería suficiente para crear la superclase Usuario?
No. Los atributos repetidos no son suficientes. La herencia expresa una relación “es-un”: la subclase hereda también el significado y las obligaciones de la superclase, y debe poder usarse donde se espera la superclase.

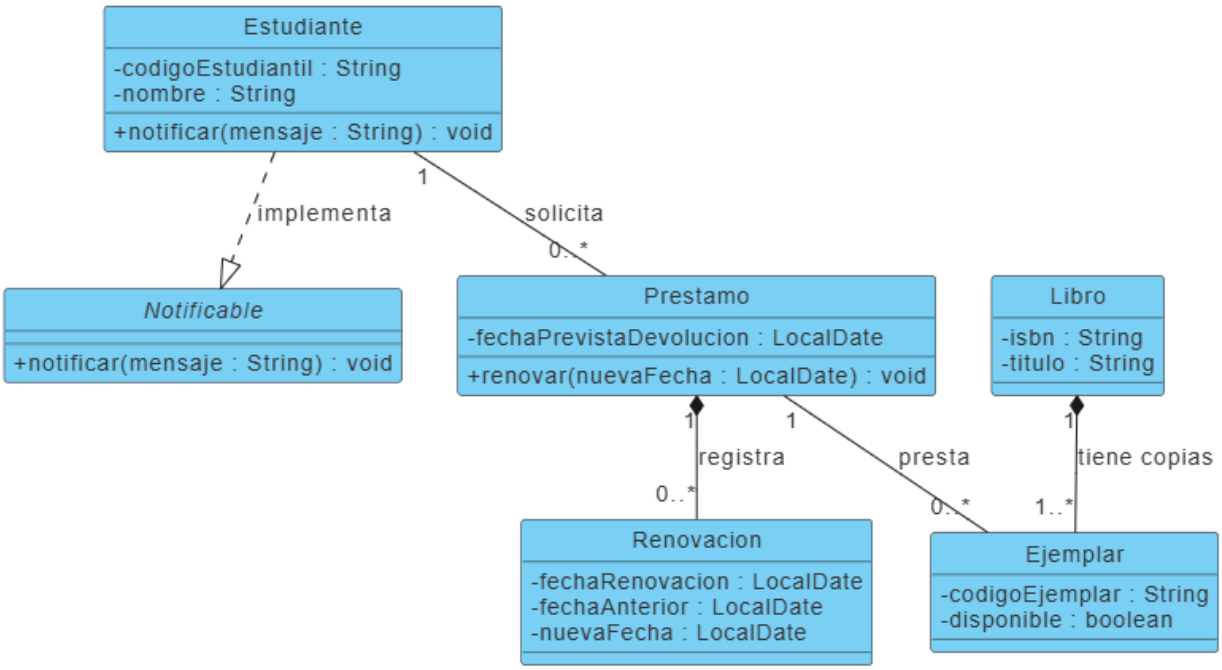
Situación en la que una superclase creada solo para evitar atributos repetidos sería conceptualmente débil
El catálogo registra **Autor** con nombre y correo (para contactar editoriales). Como coincide con Usuario, alguien podría hacer *Autor extends Usuario*. El resultado es débil: cualquier método que reciba un Usuario como por ejemplo, un préstamo o una reserva, aceptaría a un autor que ni siquiera tiene cuenta en la biblioteca.

Actividad 4 – Interfaces: expresar un contrato

¿Qué clases implementan Notificable?
Estudiante implementa Notificable. Es quien tiene préstamos y reservas, así que es quien debe enterarse de una renovación, de un vencimiento próximo o de que su reserva ya está disponible.

Bibliotecario no implementa Notificable en este bloque**.** Ningún requisito actual exige avisarle algo: él es quien ejecuta las operaciones, no quien las recibe. Agregar el método solo por si acaso obligaría a implementar un comportamiento sin uso. Si más adelante apareciera el requisito (por ejemplo, alertas de préstamos vencidos), basta con que Bibliotecario implemente la interfaz.

UML:



Garantiza que cualquier objeto Notificable acepta el mensaje notificar(String): quien lo use puede enviarle una notificación sin conocer su clase concreta. No especifica el canal (consola, correo, push, SMS), el formato, si el envío es inmediato o diferido, si se reintenta en caso de fallo, ni si el objeto guarda un historial. Todo eso es decisión de cada clase que implementa.

Por qué una interfaz no equivale a una superclase
Una superclase expresa identidad (es un tipo de) y transmite estado y comportamiento a sus subclases; en Java solo se puede heredar de una. Una interfaz expresa una capacidad (puede hacer) sin imponer estado ni implementación, y una clase puede implementar varias.

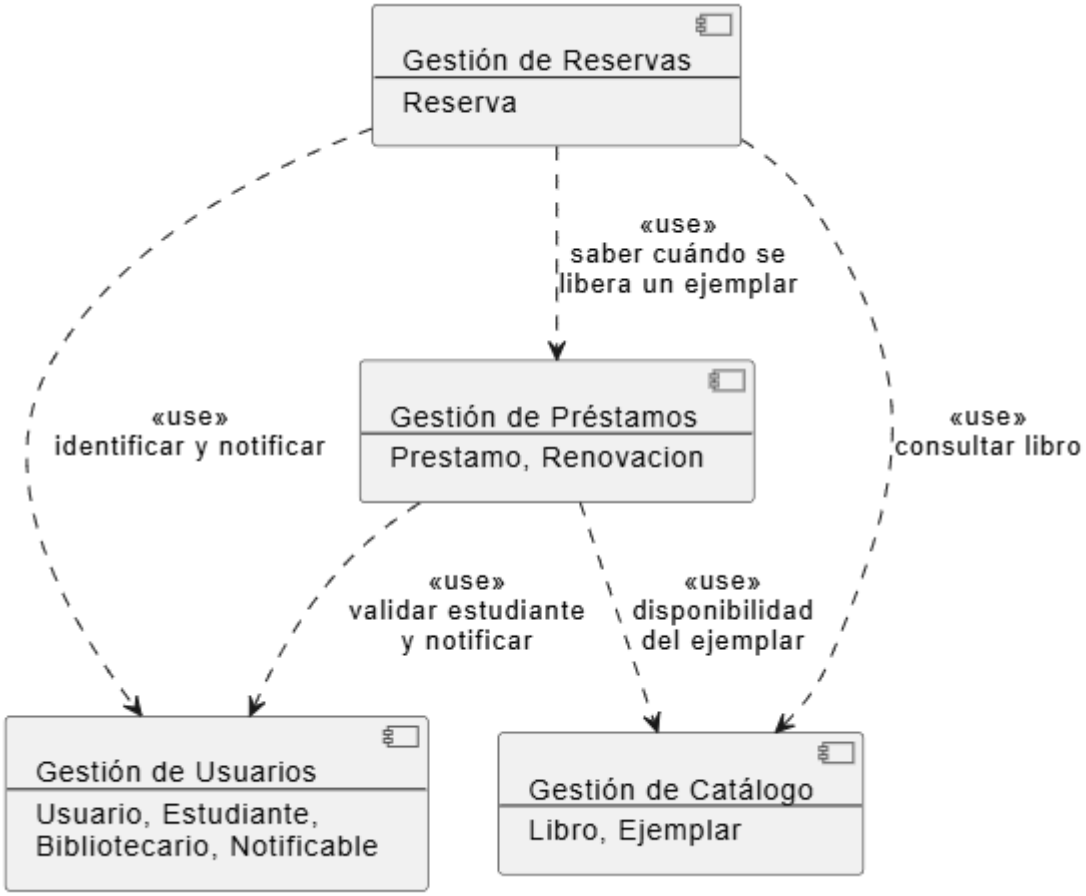
Pregunta	Respuesta del equipo
¿Qué promete una clase que implementa Notificable?	Que sabe recibir una notificación: ofrece el método público notificar(String mensaje) y se compromete a atender ese mensaje.
¿La interfaz define cómo se envía el mensaje?	No. Solo define la firma de la operación. El cómo (consola, correo, push, SMS) lo decide cada implementación; en nuestro código Estudiante lo simula imprimiendo en consola.
¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo? Explique.	Sí: public class Estudiante extends Usuario implements Notificable. Java permite una sola superclase pero varias interfaces. La herencia le da la identidad de Usuario (identificación, nombre, correo) y la interfaz le añade la capacidad de ser notificado; son dimensiones distintas y no entran en conflicto.

Actividad 5 – Implementación representativa en Java

Qué debe ocurrir en la renovación inválida: el método lanza *IllegalArgumentException* con un mensaje que explica la causa; la renovación no se crea ni se almacena, la fecha prevista permanece en 2026-10-15, la cantidad de renovaciones sigue en 1 y no se envía notificación al estudiante, porque la llamada a *notificar* queda después de la renovación dentro del bloque try. La validación ocurre antes de cualquier cambio de estado, así que el préstamo nunca queda en un estado inconsistente.

Actividad 6 – Vista funcional mediante componentes UML

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?
Gestión de Usuarios	Usuario, Estudiante, Bibliotecario, Notificable	No depende de otros componentes: es proveedor. Ofrece a los demás la identificación de usuarios y la capacidad de notificarlos.
Gestión de Catálogo	Libro, Ejemplar	No depende de otros componentes: es proveedor. Ofrece la consulta de libros y la disponibilidad de sus ejemplares.
Gestión de Préstamos	Prestamo, Renovacion	De Usuarios: validar que el estudiante existe y notificarle renovaciones o vencimientos. De Catálogo: verificar que el ejemplar esté disponible y marcarlo prestado o devuelto.
Gestión de Reservas	Reserva	De Usuarios: saber quién reserva y avisarle cuando su reserva esté lista. De Catálogo: identificar el libro reservado y si tiene ejemplares. De Préstamos: saber cuándo se devuelve un ejemplar para asignarlo a la reserva pendiente.



Conclusión

Concluimos que el diseño tiene que ver con las reglas de la biblioteca. Por ejemplo, decidir que Estudiante es un Usuario no fue porque compartieran atributos, sino porque en el sistema un estudiante realmente se comporta como un usuario. Con la interfaz Notificable entendimos que una clase puede comprometerse a hacer algo sin que importe cómo lo hace ni de quién hereda. Y al agrupar todo en componentes pudimos ver la biblioteca completa y qué partes dependen de otras. En resumen, aprendimos que diseñar no es solo crear clases, sino pensar en cómo trabajan juntas para que el sistema tenga sentido.